

Minería de Datos - Libro 1 - Clase 1 a 10

9. Árboles de Decision

Clase N° 9: Introducción a los Árboles de Decisión

Introducción

Objetivo de la Clase

Los árboles de decisión son una de las herramientas más utilizadas en minería de datos y aprendizaje supervisado debido a su interpretabilidad y facilidad de implementación. Permiten dividir datos en subconjuntos con base en condiciones sucesivas, lo que facilita la toma de decisiones automatizadas.

En esta clase aprenderemos a:

- Comprender la estructura y funcionamiento de los árboles de decisión.
- Distinguir entre clasificación y regresión con árboles de decisión.
- Explorar los principales algoritmos de construcción de árboles (ID3, C4.5, CART).
- Implementar un árbol de decisión en Python utilizando scikit-learn.
- Evaluar el rendimiento de un árbol de decisión con métricas adecuadas.

¿Por qué usar Árboles de Decisión?

Los árboles de decisión ofrecen varias ventajas, entre ellas:

- **Fácil interpretación:** Son representables visualmente, lo que facilita su comprensión.
- **Capacidad de manejar datos categóricos y numéricos.**
- **No requieren normalización de datos.**
- **Pueden manejar valores faltantes** en algunos algoritmos como C4.5.

Aplicaciones de los Árboles de Decisión

- **Medicina:** Diagnóstico de enfermedades basado en síntomas.
- **Finanzas:** Predicción de clientes con alto riesgo de impago en préstamos.
- **Marketing:** Segmentación de clientes según su comportamiento de compra.
- **Derecho:** Clasificación de casos jurídicos en categorías específicas.
- **Industria:** Predicción de fallas en maquinarias con base en mediciones sensoriales.

Ejemplo Real

Un banco quiere determinar si un cliente debe recibir un préstamo. Se pueden usar criterios como:

- Ingreso anual (¿superior a \$50,000?)
- Historial de pagos (¿ha tenido deudas impagas?)
- Edad del cliente (¿es menor de 30 años?)

El modelo toma decisiones basadas en estos atributos y asigna una clasificación: "**Aprobado**" o "**Rechazado**".

Sección 1: ¿Qué es un Árbol de Decisión?

Definición

Un árbol de decisión es una técnica de aprendizaje supervisado utilizada para clasificación y regresión, donde los datos se dividen en ramas con base en preguntas sobre sus características.

Estructura de un Árbol de Decisión

Un árbol de decisión está compuesto por:

1. **Nodo Raíz:** Representa el atributo principal sobre el cual se toma la primera decisión.
2. **Nodos Intermedios (de decisión):** Representan criterios de partición sobre diferentes atributos.
3. **Ramas:** Son las conexiones entre los nodos, representando una respuesta a la condición planteada en el nodo.
4. **Nodos Hoja:** Representan la predicción final o resultado del modelo.

Ejemplo: Un Árbol de Decisión para Aprobar un Préstamo

```

¿Ingreso > 50,000?
    /           Sí           No
   /           /           /
  /           /           /
 /           /           /
Aprobado   Rechazado

```

- Si los ingresos son mayores a \$50,000, se considera el historial crediticio.
- Si el historial es bueno, el préstamo es aprobado.
- Si cualquiera de estos factores es negativo, el préstamo es rechazado.

Ejemplo de Código en Python – Visualización de un Árbol de Decisión

```

import matplotlib.pyplot as plt
from sklearn.tree import DecisionTreeClassifier, plot_tree

# Datos ficticios de ingresos y historial crediticio
X = [[60000, 1], [35000, 0], [80000, 1], [20000, 0], [50000, 1]]
y = ["Aprobado", "Rechazado", "Aprobado", "Rechazado", "Aprobado"]

# Crear y entrenar el árbol
tree = DecisionTreeClassifier(criterion="gini", max_depth=3)
tree.fit(X, y)

# Graficar el árbol
plt.figure(figsize=(8, 5))
plot_tree(tree, feature_names=["Ingreso", "Historial"],
          class_names=["Rechazado", "Aprobado"], filled=True)
plt.show()

```

Conclusión:

- El árbol de decisión divide los datos en ramas según condiciones lógicas.
- La representación gráfica ayuda a comprender fácilmente el proceso de toma de decisiones.

1.1. Tipos de Árboles de Decisión

1. **Árboles de Clasificación:** Se utilizan cuando la variable objetivo es categórica (Ejemplo: "Sí" o "No"). Utilizan métricas como Índice Gini o Entropía para determinar las divisiones óptimas. *Ejemplo:* Diagnosticar si un paciente tiene una enfermedad (Sí/No).
2. **Árboles de Regresión:** Se utilizan cuando la variable objetivo es numérica continua. Utilizan métricas como Error Cuadrático Medio (MSE) para minimizar errores en predicciones. *Ejemplo:* Predecir el precio de una casa en función de su tamaño y ubicación.

Comparación entre Clasificación y Regresión

Tipo de Árbol	Variable Objetivo	Ejemplo
Clasificación	Categórica (Sí/No)	Diagnóstico de una enfermedad
Regresión	Continua (Números)	Predicción del precio de una casa

Ejemplo de Código en Python – Comparación de Clasificación vs. Regresión

```

import numpy as np
from sklearn.tree import DecisionTreeClassifier, DecisionTreeRegressor

# Datos simulados
X_class = np.array([[1], [2], [3], [4], [5]])
y_class = np.array(["Bajo", "Bajo", "Medio", "Alto", "Alto"])

X_reg = np.array([[1], [2], [3], [4], [5]])
y_reg = np.array([10000, 15000, 25000, 50000, 70000])

# Modelos
clf = DecisionTreeClassifier(max_depth=2).fit(X_class, y_class)
reg = DecisionTreeRegressor(max_depth=2).fit(X_reg, y_reg)

# Predicción de nuevos valores
print("Clasificación:", clf.predict([[3.5]])) # Esperado: "Medio" o "Alto"
print("Regresión:", reg.predict([[3.5]]))     # Esperado: Un valor numérico intermedio

```

Conclusión:

- Los árboles de clasificación predicen categorías discretas (Ejemplo: "Bajo", "Medio", "Alto").

- Los árboles de regresión predicen valores continuos (Ejemplo: un precio en dólares).

Ejercicio para la Sección 1

Objetivo: Comprender la diferencia entre árboles de clasificación y regresión.

Instrucciones:

1. Cargar el dataset de precios de viviendas (`sns.load_dataset("house_prices")`).
2. Entrenar un árbol de clasificación para predecir si una casa es "barata" o "cara".
3. Entrenar un árbol de regresión para predecir el precio exacto de la casa.
4. Comparar ambos enfoques y explicar sus diferencias.

Preguntas de Reflexión:

1. ¿Cuándo es más útil un árbol de clasificación en lugar de uno de regresión?
2. ¿Cómo afecta la profundidad del árbol a la precisión del modelo?
3. ¿Cómo podríamos mejorar la predicción con más atributos?

Sección 2: Construcción y Algoritmos

2.1. ¿Cómo se construye un Árbol de Decisión?

Un árbol de decisión se construye dividiendo repetidamente los datos en subconjuntos con base en preguntas sobre sus atributos.

Ejemplo: Si queremos clasificar a personas según si compran un automóvil, podríamos hacer preguntas como:

- ¿Tiene ingresos superiores a \$30,000?
- ¿Vive en una ciudad o en zona rural?
- ¿Es mayor de 25 años?

Cada pregunta divide los datos en dos subconjuntos, y este proceso continúa hasta que se llega a un nodo hoja.

Criterios para dividir los datos

Para decidir qué atributo usar en cada nodo, los algoritmos de árboles de decisión utilizan diferentes métricas:

Métrica	Uso	Descripción
Entropía	ID3 y C4.5	Mide la incertidumbre en los datos antes y después de una división.
Ganancia de Información	ID3 y C4.5	Calcula cuánto reduce la entropía una partición.
Índice Gini	CART	Mide la pureza de los nodos después de una partición.
Error Cuadrático Medio (MSE)	Árboles de Regresión	Minimiza la diferencia entre los valores reales y predichos.

Ejemplo: Cálculo de Entropía y Gini

Si tenemos 10 ejemplos y 7 pertenecen a la clase "Sí" y 3 a la clase "No", podemos calcular:

- **Entropía:** $H = -0.7 \cdot \log_2(0.7) - 0.3 \cdot \log_2(0.3) \approx 0.88$ — Una entropía de 0.88 indica que el conjunto es relativamente mixto (incierto).
- **Índice Gini:** $Gini = 1 - (0.7^2 + 0.3^2) = 1 - (0.49 + 0.09) \approx 0.42$ — Un índice Gini de 0.42 indica que el conjunto tiene diversidad de clases.

En general, cuanto más baja es la Entropía o el Gini, mejor es la partición.

2.2. Algoritmos Populares para Construcción de Árboles

1. **ID3 (Iterative Dichotomiser 3):** Utiliza Ganancia de Información basada en Entropía para seleccionar los mejores atributos. *Limitaciones:* No maneja atributos numéricos ni valores faltantes. *Ejemplo de Aplicación:* Clasificación de correos en spam y no spam.
2. **C4.5:** Es una mejora de ID3 que puede manejar atributos numéricos y datos faltantes. Utiliza Índice de Ganancia, que ajusta la ganancia de información para evitar sesgos hacia atributos con muchos valores. *Ejemplo de Aplicación:* Diagnóstico médico basado en síntomas.
3. **CART (Classification and Regression Trees):** Utiliza Índice Gini en clasificación y MSE en regresión. Solo genera árboles binarios (cada nodo tiene exactamente dos ramas). *Ejemplo de Aplicación:* Predicción de precios de viviendas.

Comparación de Algoritmos

Algoritmo	Criterio de División	Soporta Atributos Numéricos	Ramas Múltiples
ID3	Ganancia de Información (Entropía)	No	Sí
C4.5	Índice de Ganancia	Sí	Sí
CART	Gini / MSE	Sí	No (Binario)

2.3. Implementación en Python: Comparación de Algoritmos

Ejemplo: Comparar ID3 y CART en Clasificación

```
import pandas as pd
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, classification_report

# Cargar dataset Titanic
df = sns.load_dataset("titanic").dropna()

# Preprocesamiento
df["sex"] = df["sex"].map({"male": 1, "female": 0})
X = df[["age", "fare", "sex", "class"]]
y = df["survived"]

# Convertir "class" en variable numérica
X = pd.get_dummies(X, drop_first=True)

# Dividir en entrenamiento y prueba
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Árbol ID3 (simulado con entropy en scikit-learn)
tree_id3 = DecisionTreeClassifier(criterion="entropy", max_depth=3, random_state=42)
tree_id3.fit(X_train, y_train)
y_pred_id3 = tree_id3.predict(X_test)

# Árbol CART (scikit-learn usa Gini por defecto)
tree_cart = DecisionTreeClassifier(criterion="gini", max_depth=3, random_state=42)
tree_cart.fit(X_train, y_train)
y_pred_cart = tree_cart.predict(X_test)

# Evaluación de precisión
print("Precisión ID3 (Entropía):", accuracy_score(y_test, y_pred_id3))
print("Precisión CART (Gini):", accuracy_score(y_test, y_pred_cart))
```

Conclusión:

- ID3 usa Entropía y suele generar árboles más grandes y detallados.
- CART usa Gini y tiende a ser más eficiente en árboles binarios.

Ejercicio para la Sección 2

Objetivo: Comparar ID3 y CART en un dataset real.

Instrucciones:

1. Cargar el dataset `sns.load_dataset("penguins")`.
2. Aplicar un árbol ID3 (`criterion="entropy"`) para predecir la especie del pingüino.
3. Aplicar un árbol CART (`criterion="gini"`) con los mismos datos.
4. Comparar las precisiones de ambos modelos.

Pista para la solución en Python:

```
import pandas as pd
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

# Cargar dataset de pingüinos
df = sns.load_dataset("penguins").dropna()

# Preprocesamiento
X = df[["bill_length_mm", "bill_depth_mm", "flipper_length_mm", "body_mass_g"]]
y = df["species"]

# Dividir en entrenamiento y prueba
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Entrenar modelos
tree_id3 = DecisionTreeClassifier(criterion="entropy", max_depth=4, random_state=42)
tree_id3.fit(X_train, y_train)
tree_cart = DecisionTreeClassifier(criterion="gini", max_depth=4, random_state=42)
tree_cart.fit(X_train, y_train)

# Evaluar modelos
print("Precisión ID3:", accuracy_score(y_test, tree_id3.predict(X_test)))
print("Precisión CART:", accuracy_score(y_test, tree_cart.predict(X_test)))
```

Preguntas de Reflexión:

1. ¿Qué algoritmo fue más preciso en este dataset?
2. ¿Cómo influye la profundidad máxima del árbol en la precisión?
3. ¿Cómo podríamos mejorar estos modelos?

Sección 3: Evaluación y Optimización de Árboles de Decisión

Objetivo: Aprender a evaluar la calidad de un árbol de decisión y explorar técnicas para optimizar su desempeño.

3.1. ¿Cómo Evaluar un Árbol de Decisión?

Para Árboles de Clasificación:

- **Precisión (Accuracy):** Proporción de predicciones correctas.
- **Matriz de Confusión:** Muestra los verdaderos positivos, falsos positivos, verdaderos negativos y falsos negativos.
- **Precision, Recall y F1-score:** Métricas clave para evaluar la calidad de la clasificación.
- **Curva ROC-AUC:** Indica la capacidad del modelo para separar las clases.

Para Árboles de Regresión:

- **Coefficiente de Determinación (R^2):** Indica qué porcentaje de la variabilidad en los datos es explicada por el modelo.
- **Error Cuadrático Medio (MSE):** Mide el promedio de los errores al cuadrado.
- **Error Absoluto Medio (MAE):** Mide el error medio en términos absolutos.

Actividad: Evaluación de un Árbol de Clasificación en Python

```
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
import seaborn as sns
import matplotlib.pyplot as plt

# Evaluación del modelo
y_pred = tree_cart.predict(X_test)
print("Precisión del Árbol de Decisión:", accuracy_score(y_test, y_pred))
print("\nMatriz de Confusión:\n", confusion_matrix(y_test, y_pred))
print("\nReporte de Clasificación:\n", classification_report(y_test, y_pred))

# Visualización de la matriz de confusión
plt.figure(figsize=(6,4))
sns.heatmap(confusion_matrix(y_test, y_pred), annot=True, fmt="d", cmap="Blues",
            xticklabels=y.unique(), yticklabels=y.unique())
plt.xlabel("Predicción")
plt.ylabel("Realidad")
plt.title("Matriz de Confusión")
plt.show()
```

Conclusión:

- La matriz de confusión ayuda a visualizar los errores del modelo.
- Precisión y F1-score son útiles para evaluar el desempeño en clasificación.

3.2. Problemas Comunes en Árboles de Decisión

1. **Sobreajuste (Overfitting):** Ocurre cuando el árbol es demasiado profundo y aprende patrones específicos del dataset de entrenamiento, reduciendo su capacidad de generalizar. *Solución:* Limitar la profundidad máxima (`max_depth`), el número mínimo de observaciones en un nodo (`min_samples_split`) y usar poda.
2. **Subajuste (Underfitting):** Ocurre cuando el árbol es demasiado simple y no capta bien las relaciones en los datos. *Solución:* Aumentar la profundidad del árbol o incluir más variables predictoras.
3. **Selección de Atributos No Relevantes:** Si el árbol usa atributos irrelevantes, puede generar predicciones inconsistentes. *Solución:* Aplicar selección de características para elegir solo las más importantes.
4. **Sensibilidad a los Datos Desequilibrados:** Si hay clases desbalanceadas, el árbol puede favorecer la clase mayoritaria. *Solución:* Aplicar técnicas como balanceo de datos o ponderación de clases.

3.3. Optimización de Árboles de Decisión

1. Ajuste de Parámetros (Hyperparameter Tuning)

Se pueden optimizar los parámetros del árbol para evitar sobreajuste o subajuste:

Parámetro	Descripción	Cómo Ajustarlo
<code>max_depth</code>	Profundidad máxima del árbol	Reducir si hay sobreajuste
<code>min_samples_split</code>	Mínimo de muestras necesarias para dividir un nodo	Aumentar para evitar ramas pequeñas
<code>min_samples_leaf</code>	Mínimo de muestras en una hoja	Evita hojas con pocos datos
<code>max_features</code>	Número de atributos considerados en cada división	Reduce la complejidad del árbol

Ejemplo: Ajuste Manual de Parámetros

```
tree_optimized = DecisionTreeClassifier(max_depth=5, min_samples_split=10,
                                       min_samples_leaf=5, random_state=42)
tree_optimized.fit(X_train, y_train)
y_pred_opt = tree_optimized.predict(X_test)
print("Precisión optimizada:", accuracy_score(y_test, y_pred_opt))
```

2. Búsqueda de Parámetros con GridSearchCV

Se puede automatizar la búsqueda de los mejores parámetros con GridSearchCV.

```
from sklearn.model_selection import GridSearchCV

# Definir parámetros
param_grid = {
    "max_depth": [3, 5, 10],
    "min_samples_split": [2, 5, 10],
    "min_samples_leaf": [1, 5, 10]
}

# Aplicar GridSearchCV
grid_search = GridSearchCV(DecisionTreeClassifier(), param_grid, cv=5, scoring="accuracy")
grid_search.fit(X_train, y_train)
print("Mejores parámetros:", grid_search.best_params_)
```

3. Poda de Árboles

- **Poda previa (Pre-pruning):** Limita la profundidad del árbol durante su construcción (`max_depth`).
- **Poda posterior (Post-pruning):** Permite que el árbol crezca completamente y luego elimina ramas innecesarias.

Actividad extensiva: Poda de un Árbol

```
from sklearn.tree import export_text

# Mostrar el árbol antes de poda
print(export_text(tree_cart, feature_names=X.columns.tolist()))

# Aplicar poda reduciendo max_depth
tree_pruned = DecisionTreeClassifier(max_depth=3, random_state=42)
tree_pruned.fit(X_train, y_train)

# Mostrar el árbol después de la poda
print(export_text(tree_pruned, feature_names=X.columns.tolist()))
```

Conclusión:

- Ajustar `max_depth`, `min_samples_split` y `min_samples_leaf` mejora la generalización del modelo.
- GridSearchCV permite encontrar la mejor configuración automáticamente.
- La poda de árboles evita que el modelo aprenda ruido en los datos.

Actividad para la Sección 3

Objetivo: Optimizar un Árbol de Decisión para mejorar su desempeño.

Instrucciones:

1. Cargar el dataset de "diabetes" (`sns.load_dataset("diabetes")`).
2. Entrenar un árbol de decisión estándar y medir su precisión.
3. Optimizar `max_depth` y `min_samples_split` usando GridSearchCV.
4. Comparar el desempeño del modelo original vs. el optimizado.

Pista para la solución en Python:

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split

# Cargar dataset
df = sns.load_dataset("diabetes").dropna()

# Seleccionar variables
X = df.drop(columns=["outcome"])
y = df["outcome"]

# Dividir en entrenamiento y prueba
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Entrenar árbol básico
tree = DecisionTreeClassifier(random_state=42)
tree.fit(X_train, y_train)
print("Precisión Árbol Básico:", accuracy_score(y_test, tree.predict(X_test)))

# Optimizar con GridSearchCV
grid_search = GridSearchCV(tree, param_grid, cv=5, scoring="accuracy")
grid_search.fit(X_train, y_train)

# Evaluar modelo optimizado
best_tree = grid_search.best_estimator_
print("Precisión Árbol Optimizado:", accuracy_score(y_test, best_tree.predict(X_test)))
```

Preguntas de Reflexión:

1. ¿Cómo cambió la precisión después de la optimización?
2. ¿Qué parámetros fueron más importantes en la mejora del modelo?
3. ¿Cómo afecta la poda al rendimiento del árbol?

Sección 4: Ejercicio Integrador

Objetivo: Aplicar los conceptos aprendidos sobre Árboles de Decisión en un caso práctico, optimizar su rendimiento y compartir los resultados en el foro.

Ejercicio Integrador: Predicción de Riesgo Crediticio

Contexto: Un banco quiere construir un modelo para predecir si un cliente incumplirá en el pago de un crédito. Los datos incluyen información sobre ingresos, edad, historial crediticio y monto del préstamo solicitado.

Dataset para el ejercicio

Variable	Descripción	Tipo
Edad	Edad del cliente	Númerica
Ingresos (\$USD)	Ingresos anuales en dólares	Númerica
Historial	Cantidad de deudas previas	Númerica
Monto Crédito	Monto del préstamo solicitado	Númerica
Incumplimiento	1 = No pagó el préstamo, 0 = Sí pagó	Categorica (Sí/No)

Tareas a Realizar:

1. **Preprocesamiento de Datos:** Normalizar las variables numéricas. Dividir en entrenamiento (80%) y prueba (20%).
2. **Entrenar un Árbol de Decisión Básico:** Utilizar `DecisionTreeClassifier` con hiperparámetros por defecto. Evaluar el modelo con Precisión y Matriz de Confusión.
3. **Optimizar el Modelo:** Ajustar `max_depth`, `min_samples_split` y `min_samples_leaf` usando GridSearchCV. Evaluar la mejora en precisión y sobreajuste.
4. **Comparar con otro Modelo (Regresión Logística o Random Forest):** Entrenar un Random Forest y comparar sus métricas con el árbol de decisión.
5. **Interacción con una Herramienta de IA (ChatGPT, Gemini, Copilot):** Consultar cómo mejorar el modelo y comparar la respuesta con lo aprendido. *Ejemplo de consulta:* "Estoy entrenando un árbol de decisión para predecir incumplimiento de crédito. ¿Qué técnicas de optimización recomendarías?"

4.2. Implementación en Python

Código Base para la Solución


```

import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

# Simulación de datos de clientes
data = {
    "Edad": np.random.randint(20, 65, 500),
    "Ingresos": np.random.randint(20000, 120000, 500),
    "Historial": np.random.randint(0, 5, 500),
    "Monto_Credito": np.random.randint(5000, 50000, 500),
    "Incumplimiento": np.random.choice([0, 1], size=500, p=[0.7, 0.3]) # 30% de incumplimiento
}
df = pd.DataFrame(data)

# Dividir en entrenamiento y prueba
X = df.drop(columns=["Incumplimiento"])
y = df["Incumplimiento"]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Entrenar Árbol de Decisión Básico
tree = DecisionTreeClassifier(random_state=42)
tree.fit(X_train, y_train)

# Predicciones y evaluación
y_pred = tree.predict(X_test)
print("Precisión Árbol de Decisión:", accuracy_score(y_test, y_pred))
print("\nMatriz de Confusión:\n", confusion_matrix(y_test, y_pred))
print("\nReporte de Clasificación:\n", classification_report(y_test, y_pred))

# Optimización con GridSearchCV
param_grid = {
    "max_depth": [3, 5, 10],
    "min_samples_split": [2, 5, 10],
    "min_samples_leaf": [1, 5, 10]
}
grid_search = GridSearchCV(DecisionTreeClassifier(random_state=42), param_grid, cv=5, scoring="accuracy")
grid_search.fit(X_train, y_train)

# Evaluar Árbol Optimizado
best_tree = grid_search.best_estimator_
y_pred_opt = best_tree.predict(X_test)
print("\nPrecisión Árbol Optimizado:", accuracy_score(y_test, y_pred_opt))

# Comparación con Random Forest
rf = RandomForestClassifier(n_estimators=100, random_state=42)
rf.fit(X_train, y_train)
y_pred_rf = rf.predict(X_test)
print("\nPrecisión Random Forest:", accuracy_score(y_test, y_pred_rf))

```

Conclusión:

- El Árbol de Decisión optimizado debe superar al modelo básico.
- El Random Forest podría mejorar la predicción, pero con mayor complejidad computacional.

ACTIVIDAD OBLIGATORIA: VER TAREA PARA EL ENUNCIADO DE LA ACTIVIDAD.

Bibliografía

1. Breiman, L., Friedman, J., Olshen, R., & Stone, C. (1984). *Classification and Regression Trees*. CRC Press.
2. Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. Springer.
3. James, G., Witten, D., Hastie, T., & Tibshirani, R. (2013). *An Introduction to Statistical Learning: With Applications in R*. Springer.
4. Han, J., Kamber, M., & Pei, J. (2012). *Data Mining: Concepts and Techniques*. Elsevier.
5. Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., ... & Duchesnay, É. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
6. Seabold, S., & Perktold, J. (2010). Statsmodels: Econometric and Statistical Modeling with Python. *SciPy Conference*.
7. Scikit-learn Developers. (n.d.). *Scikit-learn: Machine Learning in Python*. Recuperado de <https://scikit-learn.org/>
8. Seaborn Documentation. (n.d.). *Seaborn: Statistical Data Visualization*. Recuperado de <https://seaborn.pydata.org/>

